Ionic + React

Les réponses courtes à donner, avec les fichiers à ouvrir. Chemins relatifs à projet/movie-match.

REACT : ÉTAT ET HOOKS

useState : où et pourquoi ?
Home.tsx : query, movies, loading, error. Un setter déclenche un nouveau rendu ; une variable locale ne suffit pas.

useEffect : quand charger ?
Home : [tab, query, retry]. Chaque changement relance le chargement. Le nettoyage ignore les anciennes réponses avec active = false.

Pourquoi AbortController ?
Il annule les appels backend de la page. Les appels TMDB partagés restent en cache ; active empêche leur ancien résultat d'écraser le nouveau.

useRef : quelle différence ?
CameraCapture : vidéo, flux et compteur. AuthContext : userRef et IDs en attente. .current persiste sans provoquer de rendu.

useAuth : que partage-t-il ?
AuthContext.tsx : user, login, register, logout, toggleFavorite. AuthProvider englobe le routeur ; useAuth utilise useContext.

Comment fonctionnent les favoris ?
Grille → contexte → changement immédiat → POST/DELETE. Échec : restauration de cet ID seulement. Les doubles clics sont bloqués.

Pourquoi des props et des callbacks ?
MovieGrid reçoit movies et onError. CameraCapture transmet la photo avec onPhoto. L'état partagé reste dans le parent ou le contexte.

Formulaires et navigation ?
Login/Register : value + onChange, preventDefault(), puis useNavigate. Après succès : /tabs/home avec replace: true.

IONIC : ÉCRANS ET CAMÉRA

Qui fait quoi ?
React : état et rendu. Ionic : composants et navigation. Capacitor : accès natif. MongoDB : données via Express.

Où l'application démarre-t-elle ?
main.tsx : setupIonicReact() puis createRoot. App.tsx : IonApp, AuthProvider, IonReactRouter et IonRouterOutlet.

Quels composants montrer ?
IonPage = page ; IonContent = zone défilante ; IonHeader/Toolbar/Title = en-tête ; IonSpinner = attente ; IonAvatar/Badge = profil et score.

Utilise-t-on IonTabs ?
Non. Tabs.tsx contient un outlet imbriqué et des NavLink personnalisés. Tabs.scss passe de navigation basse à barre latérale à 900 px.

Comment rechercher un film ?
Home.tsx : IonSearchbar, onIonInput, e.detail.value et debounce={300}. setQuery relance l'effet ; movies alimente MovieGrid.

useEffect([]) = chaque visite ?
Non : Ionic peut garder la page montée. CameraCapture utilise aussi useIonViewWillLeave pour arrêter la caméra au départ.

Comment la photo est-elle prise ?
Natif : Camera.takePhoto. Web : getUserMedia → vidéo → canvas → JPEG data URL. Aucun input fichier. Capture, annulation ou départ arrêtent le flux.

Où changer l'aspect et la caméra ?
theme/variables.scss : couleurs. pages/*.scss : écrans. CameraCapture.tsx : qualité native 70 et taille 300 ; web JPEG 0.7 et taille max. 300.

API + changements rapides

Backend : movie-match-api. Les snippets sont des exercices, pas des modifications déjà appliquées.

Deux API : TMDB fournit les films. Express + MongoDB conserve comptes, favoris et catalogue. **api()** ajoute URL backend + JWT (sauf login/register), délai 12 s. **requestJson()** fait le fetch générique, délai 10 s par défaut.

APPEL DEPUIS LE FRONTEND	OÙ / DONNÉES
POST /auth/login ; POST /auth/register	AuthContext.tsx → réponse { token, user }. Session dans les clés localStorage token et user.
GET /movies ; POST /movies	getCatalog() / Admin.add. Ajout admin : tmdbId, title, overview, posterPath, releaseDate, voteAverage.
POST ou DELETE /users/favorites/:id	AuthContext.toggleFavorite. ID TMDB numérique, pas _id MongoDB.
GET /users ; PATCH /users/:id/toggle-active	Admin.tsx. active est inversé ; le compte reste en base.
GET /matches?threshold=75	Matches.tsx → controller backend → { user, score, common }.

SI LE PROFESSEUR DEMANDE DE CHANGER…

DEMANDE	MODIFICATION PRÉCISE
Adresse du backend	.env.local : VITE_API_URL=https://serveur/api ; redémarrer Vite. api('/movies') ajoute ce préfixe.
Populaires → mieux notés	lib/movies.ts, getPopular() : /movie/top_rated et clé cache top-rated. Renommer le texte dans Home.
GET → POST avec JSON	api('/route', { method: 'POST', body: JSON.stringify({ query }) }). Le serveur doit accepter cette route/méthode.
Réponse { movies: [...] }	getCatalog() : lire response.movies avant le map. Adapter le type et le serveur si le contrat change.
Un champ de plus	Vérifier le JSON → types.ts → transformation dans movies.ts → JSX. Un type ne crée pas une donnée absente.
Seuil à 80 %	Matches.tsx : query + textes. Backend : minimum/default si 80 doit être obligatoire. Garder score > threshold.
Recherche plus lente / nouvelle page	Home et Admin : debounce={500}. Nouvelle page : IonPage + route et lien dans Tabs.tsx.
Toujours d'anciens films	Cache TMDB : 10 min, 100 entrées par Map. clearMovieCache() vide les deux Maps ; la clé doit inclure les paramètres variables.

Pièges à retenir : Jaccard = communs / union × 100 ; 75 exact est refusé. L'API filtre avant toFixed(2). Un rôle local ne donne pas les droits serveur : middleware auth + admin. HTTP 400/403 est traité via response.ok, pas automatiquement rejeté par fetch.

Démonstration : frontend 4200, backend 3000. Si login échoue : API + MONGO_URI/JWT_SECRET/JWT_EXPIRES. Si caméra échoue : permission + HTTPS/localhost + appareil. Vérifier : npm test ; npm run lint ; npm run build (sortie www).